

[Lecture Notebook] Evaluating Predictive Performance

MG3701 Data Mining, Spring 2025

Chad (Chungil Chae)

Sun, 25 May 2025

Table 5.1

```
# package forecast is required to evaluate performance
library(forecast)
```

Registered S3 method overwritten by 'quantmod':

```
method          from
as.zoo.data.frame zoo
```

Registered S3 methods overwritten by 'forecast':

```
method from
head.ts stats
tail.ts stats
```

```
# load file
toyota.corolla.df <- read.csv("data/ToyotaCorolla.csv")

# randomly generate training and validation sets
training <- sample(toyota.corolla.df$Id, 600)
validation <- sample(setdiff(toyota.corolla.df$Id, training), 400)

# run linear regression model
reg <- lm(Price~., data=toyota.corolla.df[, -c(1,2,8,11)], subset=training,
         na.action=na.exclude)
pred_t <- predict(reg, na.action=na.pass)
pred_v <- predict(reg, newdata=toyota.corolla.df[validation, -c(1,2,8,11)],
         na.action=na.pass)

## evaluate performance
# training
accuracy(pred_t, toyota.corolla.df[training,]$Price)
```

```
13           ME      RMSE      MAE      MPE      MAPE
14 Test set -5.966765e-11 1015.019 785.8117 -0.8151447 7.767709
```

```
# validation
accuracy(pred_v, toyota.corolla.df[validation,]$Price)
```

```
15           ME      RMSE      MAE      MPE      MAPE
16 Test set -91.36236 1109.289 791.5724 -1.783455 8.116831
```

17 **Figure 5.2**

```
# remove missing Price data
toyota.corolla.df <-
  toyota.corolla.df[!is.na(toyota.corolla.df[validation,]$Price),]

# generate random Training and Validation sets
training <- sample(toyota.corolla.df$Id, 600)
validation <- sample(toyota.corolla.df$Id, 400)

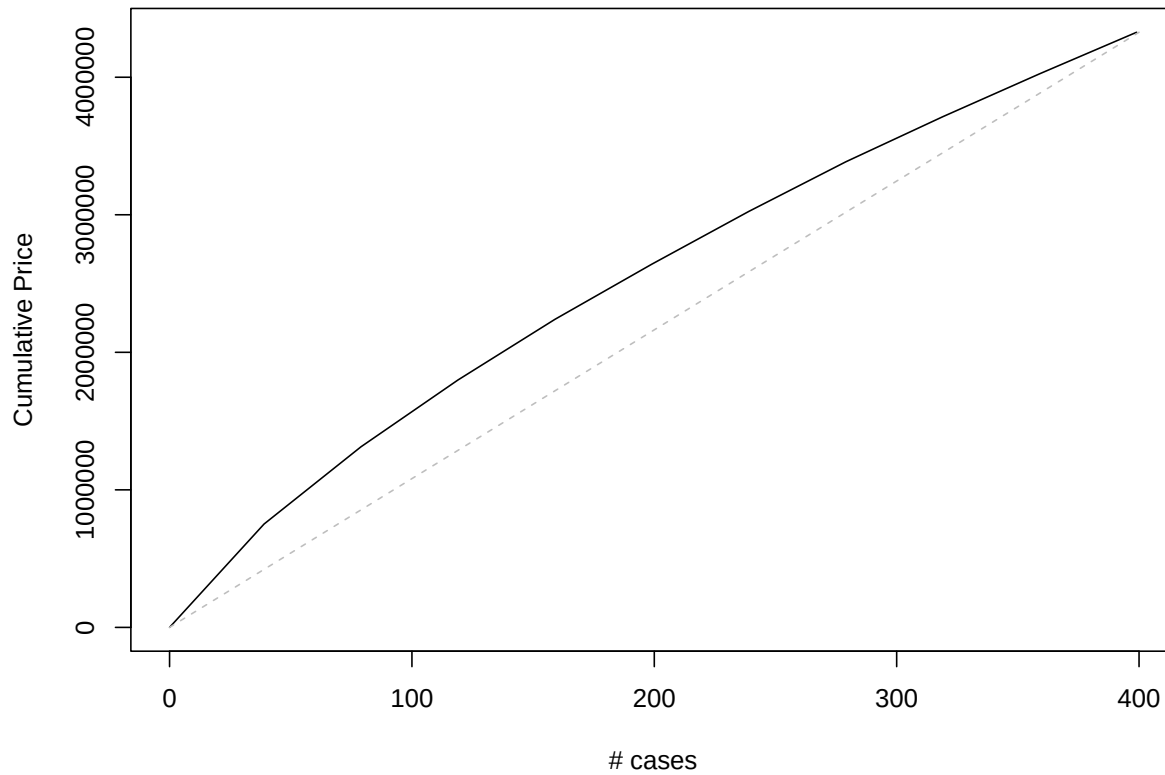
# regression model based on all numerical predictors
reg <- lm(Price~., data = toyota.corolla.df[, -c(1,2,8,11)], subset = training)

# predictions
pred_v <- predict(reg, newdata = toyota.corolla.df[validation, -c(1,2,8,11)])

# load package gains, compute gains (we will use package caret for categorical y later)
library(gains)
gain <- gains(toyota.corolla.df[validation,]$Price[!is.na(pred_v)], pred_v[!is.na(pred_v)])

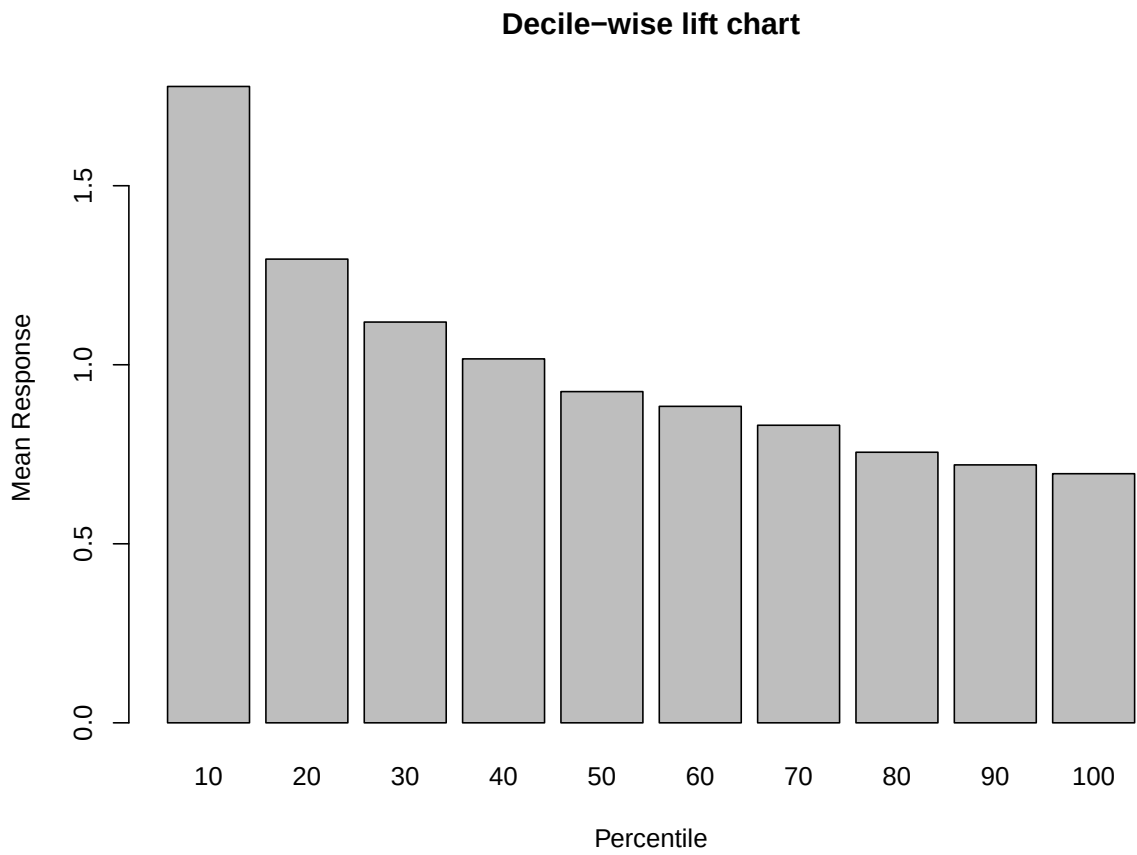
# cumulative lift chart
options(scipen=999) # avoid scientific notation
# we will compute the gain relative to price
price <- toyota.corolla.df[validation,]$Price[!is.na(toyota.corolla.df[validation,]$Price)]
plot(c(0,gain$cume.pct.of.total*sum(price))~c(0,gain$cume.obs),
     xlab="# cases", ylab="Cumulative Price", main="Lift Chart", type="l")

# baseline
lines(c(0,sum(price))~c(0,dim(toyota.corolla.df[validation,])[1]), col="gray", lty=2)
```

Lift Chart

18

```
# Decile-wise lift chart
barplot(gain$mean.resp/mean(price), names.arg = gain$depth,
        xlab = "Percentile", ylab = "Mean Response", main = "Decile-wise lift chart")
```



19

20 **Table 5.5**

```
library(caret)
```

21 Loading required package: lattice

22

23 Attaching package: 'caret'

24 The following object is masked from 'package:purrr':

25

26 lift

```
library(e1071)

owner.df <- read.csv("data/ownerExample.csv")
#confusionMatrix(as.factor(ifelse(owner.df$Probability>0.5, 'owner', 'nonowner')),
#                owner.df$Class)
#confusionMatrix(as.factor(ifelse(owner.df$Probability>0.25, 'owner', 'nonowner')),
#                owner.df$Class)
#confusionMatrix(as.factor(ifelse(owner.df$Probability>0.75, 'owner', 'nonowner')),
#                owner.df$Class)
```

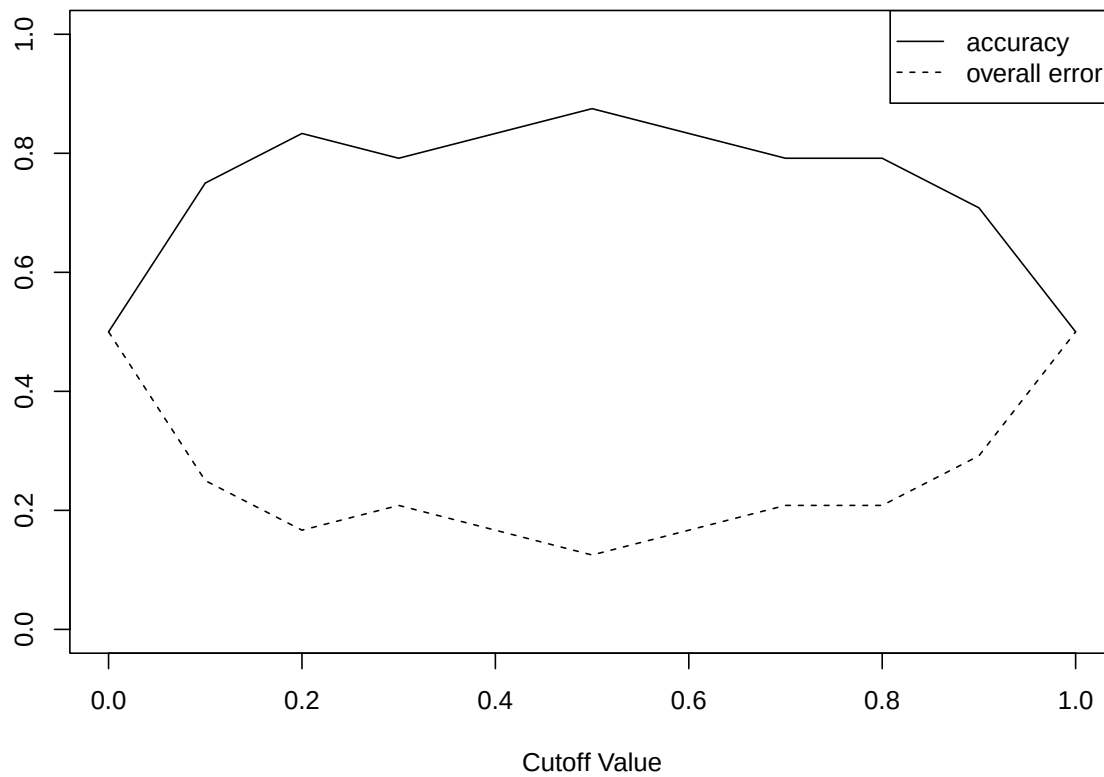
27 **Figure 5.4**

```
# replace data.frame with your own
df <- read.csv("data/liftExample.csv")

# create empty accuracy table
accT = c()

# compute accuracy per cutoff
for (cut in seq(0,1,0.1)){
  cm <- confusionMatrix(as.factor(1 * (df$prob > cut)), as.factor(df$actual))
  accT = c(accT, cm$overall[1])
}

# plot accuracy
plot(accT ~ seq(0,1,0.1), xlab = "Cutoff Value", ylab = "", type = "l", ylim = c(0, 1))
lines(1-accT ~ seq(0,1,0.1), type = "l", lty = 2)
legend("topright", c("accuracy", "overall error"), lty = c(1, 2), merge = TRUE)
```



28

29 **Figure 5.5**

```
library(pROC)
```

30 Type 'citation("pROC")' for a citation.

31

32 Attaching package: 'pROC'

33 The following objects are masked from 'package:stats':

34

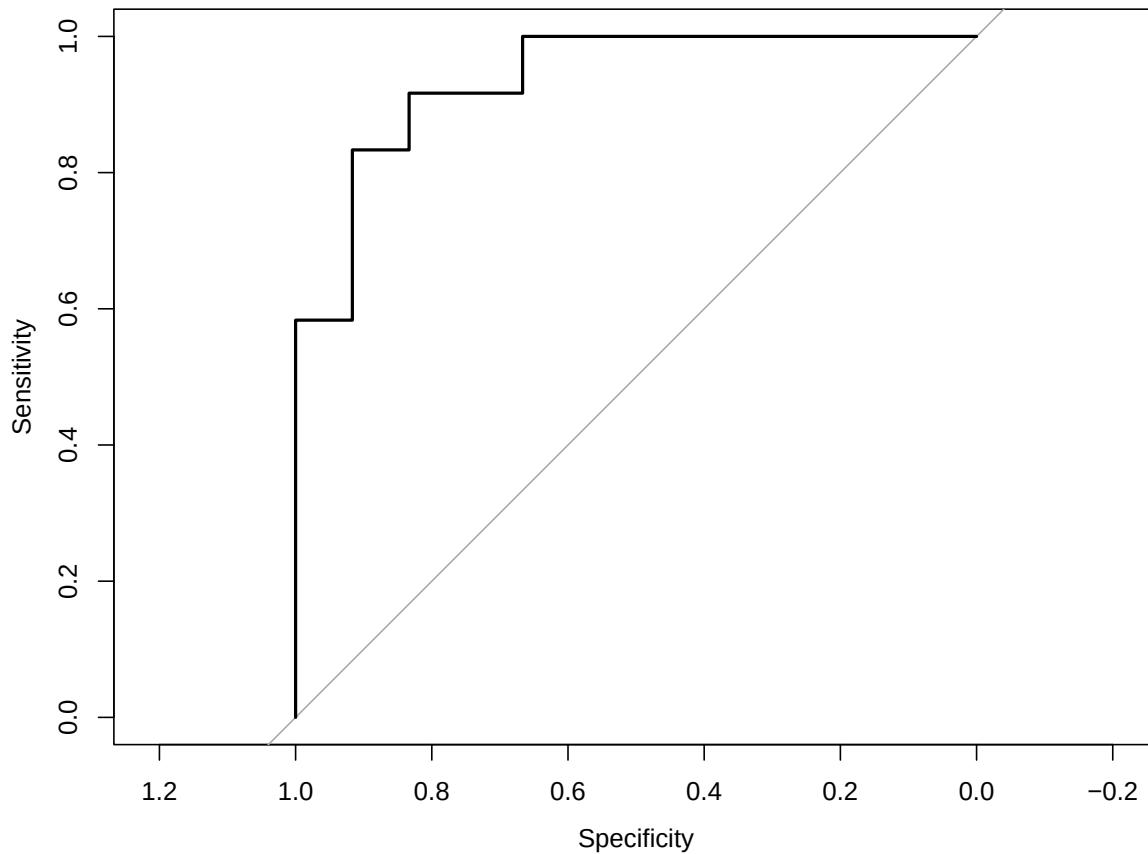
35 cov, smooth, var

```
r <- roc(df$actual, df$prob)
```

36 Setting levels: control = 0, case = 1

37 Setting direction: controls < cases

```
plot.roc(r)
```



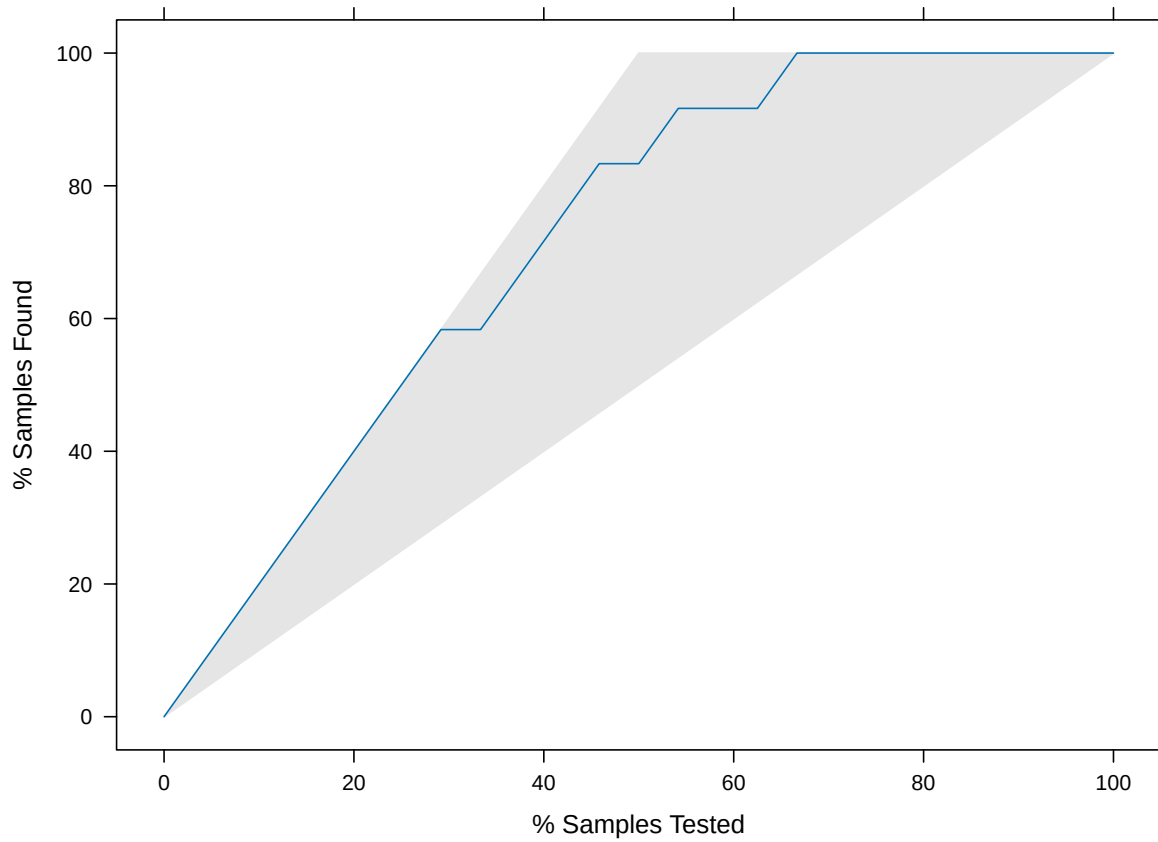
38

```
# compute auc  
auc(r)
```

39 Area under the curve: 0.9375

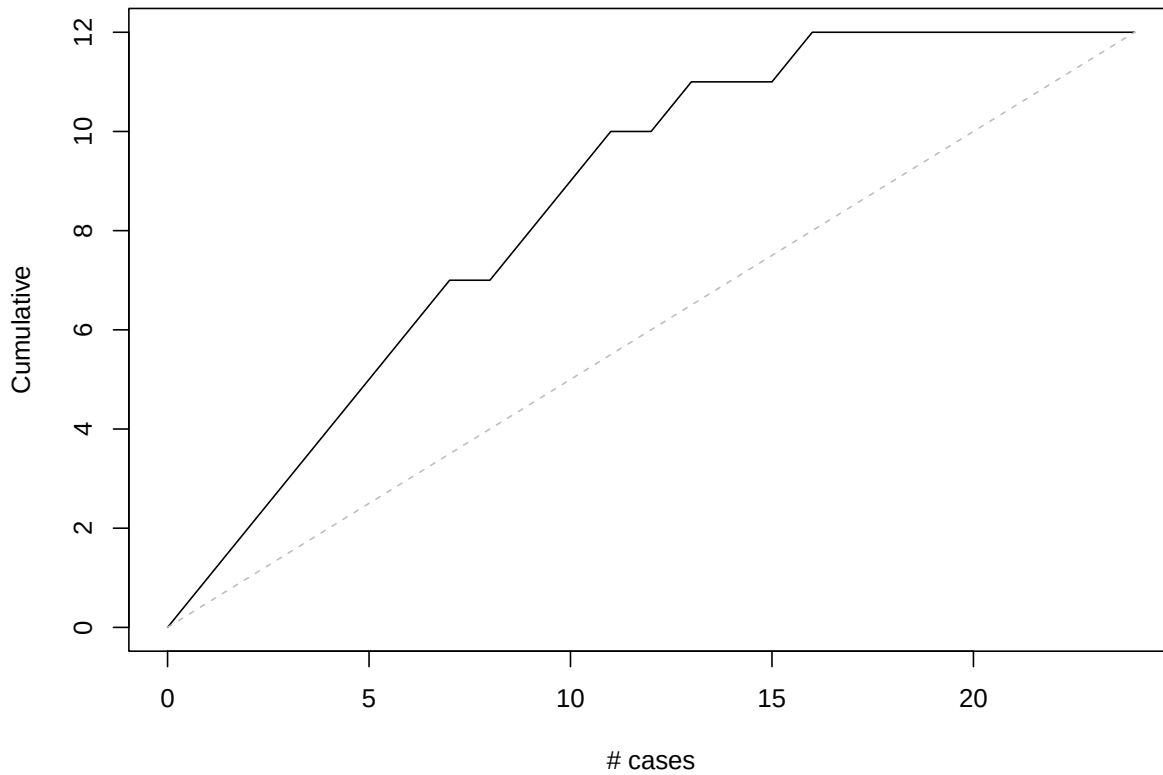
40 **Figure 5.6**

```
# first option with 'caret' library:  
library(caret)  
lift.example <- lift(relevel(as.factor(actual), ref="1") ~ prob, data = df)  
xyplot(lift.example, plot = "gain")
```



41

```
# Second option with 'gains' library:
library(gains)
df <- read.csv("data/liftExample.csv")
gain <- gains(df$actual, df$prob, groups=dim(df)[1])
plot(c(0, gain$cume.pct.of.total*sum(df$actual)) ~ c(0, gain$cume.obs),
     xlab = "# cases", ylab = "Cumulative", type="l")
lines(c(0,sum(df$actual))~c(0,dim(df)[1]), col="gray", lty=2)
```



42

43 **Figure 5.7**

```
# use gains() to compute deciles.  
# when using the caret package, deciles must be computed manually.  
  
gain <- gains(df$actual, df$prob)  
barplot(gain$mean.resp / mean(df$actual), names.arg = gain$depth, xlab = "Percentile",  
        ylab = "Mean Response", main = "Decile-wise lift chart")
```

